

Principal Component Analysis—Stepping into the details of the math

 medium.com/sho-jp/principal-component-analysis-101-part-1-d62aa2b0cc36

February 22, 2019

Today, we going to dive a little deep into a famous dimensionality reduction technique called Principal Component Analysis (PCA).

Or if you like reading texts instead, check below:

What is an intuitive explanation for PCA?

Answer (1 of 29): Imagine a big family dinner, where everybody starts asking you about PCA. First you explain it to...

www.quora.com

In short, PCA projects the data onto a low dimension by maximizing the variance (or minimizing the distance from the chosen axis) of the data in each principal component axis.

The focus of this story

Today's focus is not to explain the intuitive meaning of PCA, but rather trying to understand the math behind the calculation of PCA.

The content is mainly from Q&A discussions from the below links, so if you want to just read all of it, check below links.

What is an intuitive explanation for how PCA turns from a geometric problem (with distances) to a...

beginingroup\$ The geometric problem that PCA is trying to optimize is clear to me: PCA tries to find the first principal...

stats.stackexchange.com

PCA objective function: what is the connection between maximizing variance and minimizing error?

beginingroup\$ The PCA algorithm can be formulated in terms of the correlation matrix (assume the data X has already...

stats.stackexchange.com

That being said, let's get started.

First, let's see the common way of performing PCA using eigendecomposition.

Let's begin by defining a data matrix $\mathbf{X}$. The dimension of $\mathbf{X}$ is " n " by " f ". You could think of it like the data matrix has " n " number of samples and " f " number of features.

$$\mathbf{X} \in \mathbb{R}^{n \times f}$$

(n : number of samples, f : number of features)

Before running PCA, it's always better to center the data, because in the end, PCA is just rotating the data around to project the data on top of principal components.

$$\mathbf{X}_c = \mathbf{X} - \mu_{\mathbf{X}} \quad , \quad \mathbf{X}_c \in \mathbb{R}^{n \times f}$$

($\mu_{\mathbf{X}}$: matrix where mean of each feature of $\mathbf{X}$ is calculated)

Of course, we only subtracted the mean from each feature column, so the dimension hasn't changed.

Now, if you read how to calculate PCA, one of the common steps you see is you calculate the sample covariance matrix using the centered data.

$$\mathbf{C} = \frac{\mathbf{X}_c^T \mathbf{X}_c}{n-1} \quad , \quad \mathbf{C} \in \mathbb{R}^{f \times f}$$

Sample mean and covariance - Wikipedia

needs additional citations for verification .improve this article by adding citations to reliable sources. Unsourced...

en.wikipedia.org

And you run either eigendecomposition or singular value decomposition (I'm going to explain this version in the later part of the story).

$$C = Q \Lambda Q^T$$

$$\begin{cases} Q \in \mathbb{R}^{d \times d} & (\text{eigenvector matrix}) \\ \Lambda \in \mathbb{R}^{d \times d} & (\text{diagonal matrix with eigenvalues}) \end{cases}$$

Linear Algebra 101 — Part 7: Eigendecomposition when symmetric

Today, we are studying more advanced topics in Linear Algebra that are more relevant and useful in machine learning.

medium.com

In short, the eigenvector matrix Q is the matrix that contains principal components and Λ is the eigenvalue matrix (eigenvalues corresponding to the magnitude of PCs so you would order them from large to small so that the larger the eigenvalue is, the more important the corresponding PC is).

So to reduce the dimensionality, you would just pick a few principal components, let's say "s" of them. Now you have selected numbers of principal components matrix Q_s .

In the end, if you want to project the original data matrix onto the principal component's axes, you would just multiply the data matrix with the above principal component matrix to get the following.

$$Q_s \in \mathbb{R}^{d \times s}$$

$$\text{PC projected data}$$

$$X_c Q_s \in \mathbb{R}^{n \times s}$$

That's it! Easy!

Wait a minute...

Yeah, calculating PCA or implementing PCA yourself is not that hard. But I was always wondering how does:

"Maximizing variance along the principal components"

corresponds to

“Performing eigendecomposition of the covariance matrix”

Apparently, there are fewer articles explaining this. One of the few I found is the links I posted earlier and let me explain that here.

Suppose you have a principal component vector $\mathbf{v}$.

$$\mathbf{v} \in \mathbb{R}^{d \times 1}$$

$\hookrightarrow$ Principal Component Vector

Now, let's use that to project the data onto the PC.

PC projected data

$$\mathbf{X}_c \mathbf{v} \in \mathbb{R}^{n \times 1}$$

If this is the projected data, the variance should be maximized, right? Because that was the whole point of PCA.

So first, let's calculate the sample variance.

Estimation of sample variance

$$\begin{aligned} Q &= \frac{1}{n-1} (\mathbf{X}_c \mathbf{v})^T (\mathbf{X}_c \mathbf{v}) \\ &= \mathbf{v}^T \frac{\mathbf{X}_c^T \mathbf{X}_c}{n-1} \mathbf{v} = \mathbf{v}^T \mathbf{C} \mathbf{v} \end{aligned}$$

$$\begin{cases} \mathbf{C} : \text{Estimation of Sample Covariance} \\ \mathbf{C} \in \mathbb{R}^{d \times d} \end{cases}$$

Since the whole purpose of PCA is to maximize the variance along the projected axis, we are going to do that in the equation too. But we need to be careful. If we don't set the constraints, say have very large “ $\mathbf{v}$ ”, then we can easily maximize the variance “ Q ”. To avoid this, we are setting a constraint to make it a uniform vector.

We want to maximize variance Q
 with a constraint

$$\|v\| = v^T v = 1$$

Now, we have a problem.

1. We want to maximize “Q”
2. But at the same time, we are constrained by “”

To solve this kind of problem, people usually use Lagrange multiplier.

Lagrange Multiplier

$$L(v) = Q - \lambda (v^T v - 1)$$

$$\begin{aligned} \frac{\partial L}{\partial v} = 0 &\iff \frac{\partial}{\partial v} (v^T C v - \lambda (v^T v - 1)) = 0 \\ &\iff C v - \lambda v = 0 \\ &\therefore C v = \lambda v \end{aligned}$$

(Note that to be more precise, you need to also check other derivatives too, but we are just ignoring those here.)

Lagrange multiplier - Wikipedia

(Sometimes an additive constant is shown separately rather than being included in g, in which case the constraint is...

en.wikipedia.org

After solving the Lagrange multiplier, to maximize variance, the vector “**v**” has to be the eigenvector of the covariance matrix! This is where the idea of solving eigendecomposition is coming from. That’s why you could solve the PCA without doing some iterative optimization.

OK, so at last, what’s the difference between eigendecomposition based PCA vs singular value decomposition (SVD) based PCA?

Eigendecomposition PCA was easy to understand. Both for the proof and solving PCA itself.

However, in practice, most of the implementation is based on Singular Value Decomposition (SVD). Why is that?

If you remember how we computed eigenvectors and eigenvalues, eigendecomposition based PCA needs to calculate the covariance matrix first. The covariance matrix was “ f ” by “ f ”. If “ f ” is small, it won’t be a problem. But imagine a huge “ f ”, let’s say you are dealing with an image. “ f ” suddenly becomes a huge number, more than a million in some cases. Then computing the covariance becomes infeasible. This is where SVD based PCA can help.

Let’s see why.

Singular Value Decomposition (SVD)

$$X_c = U \Sigma V^T$$

$$\begin{cases} U \in \mathbb{R}^{n \times n} \\ \Sigma \in \mathbb{R}^{n \times f} \\ V^T \in \mathbb{R}^{f \times f} \end{cases}$$

If you remember the SVD we covered last time, the centered data can be decomposed into three different matrices. “ U ” and “ V ” are unitary matrices and “ Σ ” is a diagonal rectangular matrix.

Linear Algebra 101 — Part 9: Singular Value Decomposition (SVD)

Today, we are going to study another famous decomposition called “Singular Value Decomposition (SVD)”.

medium.com

Let’s see what happens if we follow the same logic as we did in the eigendecomposition.

$$\begin{aligned} C &= \frac{X_c^T X_c}{n-1} = \frac{V \Sigma^T U^T U \Sigma V^T}{n-1} \\ &= V \frac{|\Sigma|^2}{n-1} V^T \end{aligned}$$

Turns out, the matrix “ V ” is the eigenvector matrix we needed! Therefore, if we were to do the same and project the data on to the principal components, we can completely skip the calculation of the covariance matrix because the following equation is just the altered

equation of the SVD of the centered data matrix! (You just need to multiply the matrix “ $\mathbf{V}$ ” from the right side on $\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}'$)

PC projected data

$$\mathbf{X}_c \mathbf{V} = \mathbf{U} \mathbf{\Sigma}$$

Summary

- PCA projects the data onto the principal components where the variance of the data is maximized (or the total distances from each point to the axis is minimized)
- You can derive why PCA is all about solving eigendecomposition of the covariance matrix using Lagrange multiplier
- Practically, people use SVD to calculate PCA because of computation efficiency

That's all! Hope it helps! See you next time!